

Contexte

Dans un environnement professionnel, la gestion d'applications distribuées nécessite une orchestration efficace, sécurisée et scalable. Kubernetes est devenu l'outil incontournable pour déployer, gérer et faire évoluer ces applications en production.

Ce projet illustre mes compétences en administration système et réseau appliquées à Kubernetes. Il démontre ma capacité à concevoir une infrastructure cloud résiliente, sécurisée et automatisée, conforme aux bonnes pratiques de l'industrie.

Introduction

L'objectif de ce projet est de déployer une application backend simple, mais critique, dans un cluster Kubernetes, en utilisant exclusivement des commandes en ligne de commande (CLI).

Le projet comprend :

- La création d'un namespace dédié pour isoler les ressources
- La gestion des ressources via des quotas (ResourceQuota)
- Le déploiement d'un backend scalable (Nginx) avec gestion des ressources CPU et mémoire
- La mise en place d'un autoscaler (HPA) pour ajuster automatiquement la capacité en fonction de la charge
- La définition d'une NetworkPolicy pour sécuriser le trafic réseau interne
- La simulation d'incidents et le diagnostic via les logs et outils Kubernetes

Ce travail montre ma maîtrise des concepts fondamentaux Kubernetes, ainsi que mes compétences avancées en gestion de l'infrastructure, sécurité et automatisation.

1. Création du namespace

Le namespace est un espace logique permettant d'isoler et d'organiser les ressources Kubernetes.

```
controlplane:~$ kubectl create namespace test-portfolio
kubectl config set-context --current --namespace=test-portfolio
namespace/test-portfolio created
Context "kubernetes-admin@kubernetes" modified.
```

Ces commandes créent le namespace test-portfolio et configurent le contexte actif pour que toutes les commandes suivantes s'exécutent dans ce namespace.

2. Création du ResourceQuota

Pour limiter les ressources consommées dans le namespace, on crée un ResourceQuota via un fichier YAML.

Créer le fichier resourcequota.yaml :

```
controlplane:~$ nano resourcequota.yaml
controlplane:~$ cat resourcequota.yaml
apiVersion: v1
kind: ResourceQuota
metadata:
  name: test-portfolio-quota
spec:
  hard:
    requests.cpu: "2"
    requests.memory: 2Gi
    limits.cpu: "4"
    limits.memory: 4Gi
controlplane:~$
```

Appliquer le quota

Appliquer la limite de ressources dans Kubernetes :

```
controlplane:~$ kubectl apply -f resourcequota.yaml
resourcequota/test-portfolio-quota created
```

Déployer le backend :

```
controlplane:~$ kubectl create deployment test-portfolio-backend --image=nginx --replicas=3
kubectl set resources deployment test-portfolio-backend \
  --limits=cpu=500m,memory=256Mi \
  --requests=cpu=200m,memory=128Mi
kubectl expose deployment test-portfolio-backend --port=80
deployment.apps/test-portfolio-backend created
deployment.apps/test-portfolio-backend resource requirements updated
service/test-portfolio-backend exposed
```

Cela garantit que les pods dans test-portfolio ne dépasseront pas les ressources définies.

5. Appliquer l'autoscaler et déployer le backend

L'**autoscaler** (Horizontal Pod Autoscaler, HPA) ajuste automatiquement le nombre de pods backend en fonction de la charge CPU.

Le backend sera déployé sous forme de 3 pods répliqués, utilisant l'image Nginx.

```
controlplane:~$ kubectl autoscale deployment test-portfolio-backend --cpu-percent=50 --min=3 --max=6
kubectl get hpa
Flag --cpu-percent has been deprecated, Use --cpu with percentage or resource quantity format (e.g., '70%' for utilization or '500m' for milliCPU).
horizontalpodautoscaler.autoscaling/test-portfolio-backend autoscaled
NAME                REFERENCE                TARGETS          MINPODS   MAXPODS   REPLICAS   AGE
test-portfolio-backend  Deployment/test-portfolio-backend  cpu: <unknown>/50%   3         6         0          0s
```

- `--cpu-percent=50` : ajoute des pods si la charge moyenne CPU dépasse 50%
- `--min=3` : minimum 3 pods
- `--max=6` : maximum 6 pods

Cela permet d'optimiser la consommation et la performance sans intervention manuelle.

6. Création de la NetworkPolicy

Pour renforcer la sécurité réseau, on crée une politique réseau qui contrôle le trafic entrant et sortant.

Créer le fichier networkpolicy.yaml :

```
controlplane:~$ nano networkpolicy.yaml
controlplane:~$ kubectl apply -f networkpolicy.yaml
networkpolicy.networking.k8s.io/test-portfolio-deny-all created
```

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: test-portfolio-deny-all
spec:
  podSelector: {}
  policyTypes:
  - Ingress
  - Egress
```

Cette policy bloque tout le trafic entrant et sortant dans le namespace, servant de base pour des règles plus spécifiques.

Lister les pods :

```
controlplane:~$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
test-portfolio-backend-6d9bbdf45d-48dsz	1/1	Running	0	18m
test-portfolio-backend-6d9bbdf45d-62bbz	1/1	Running	0	18m
test-portfolio-backend-6d9bbdf45d-gmqtq	1/1	Running	0	17m

Suppression pod et une création automatique :

```
controlplane:~$ kubectl get pods -n test-portfolio
```

NAME	READY	STATUS	RESTARTS	AGE
test-portfolio-backend-6d9bbdf45d-4n7hd	1/1	Running	0	4m10s
test-portfolio-backend-6d9bbdf45d-9w72s	1/1	Running	0	3m55s
test-portfolio-backend-6d9bbdf45d-pcwz6	1/1	Running	0	4m21s

```
controlplane:~$ kubectl delete pod test-portfolio-backend-6d9bbdf45d-4n7hd -n test-portfolio
pod "test-portfolio-backend-6d9bbdf45d-4n7hd" deleted from test-portfolio namespace
controlplane:~$ kubectl get pods -n test-portfolio -w
```

NAME	READY	STATUS	RESTARTS	AGE
test-portfolio-backend-6d9bbdf45d-9w72s	1/1	Running	0	4m52s
test-portfolio-backend-6d9bbdf45d-bdgl9	1/1	Running	0	18s
test-portfolio-backend-6d9bbdf45d-pcwz6	1/1	Running	0	5m18s

Vérification :

```

controlplane:~$ kubectl get all -n test-portfolio
kubectl get hpa
kubectl get networkpolicy
kubectl get namespaces
NAME                                READY   STATUS    RESTARTS   AGE
pod/test-portfolio-backend-6d9bbdf45d-62bbz   1/1     Running   0           34m
pod/test-portfolio-backend-6d9bbdf45d-8pnd7   1/1     Running   0           14m
pod/test-portfolio-backend-6d9bbdf45d-gmqqtq   1/1     Running   0           33m

NAME                                TYPE      CLUSTER-IP   EXTERNAL-IP   PORT(S)    AGE
service/test-portfolio-backend        ClusterIP   10.101.116.151   <none>        80/TCP      34m

NAME                                READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/test-portfolio-backend  3/3     3             3           34m

NAME                                DESIRED   CURRENT   READY   AGE
replicaset.apps/test-portfolio-backend-69b558658b   0         0         0       34m
replicaset.apps/test-portfolio-backend-6d9bbdf45d   3         3         3       34m

NAME                                REFERENCE   TARGETS   MINPODS   MAXPODS   REPLICAS   AGE
horizontalpodautoscaler.autoscaling/test-portfolio-backend  Deployment/test-portfolio-backend  cpu: <unknown>/50%   3         6         3         28m
NAME                                REFERENCE   TARGETS   MINPODS   MAXPODS   REPLICAS   AGE
test-portfolio-backend              Deployment/test-portfolio-backend  cpu: <unknown>/50%   3         6         3         28m
NAME                                POD-SELECTOR  AGE
test-portfolio-deny-all             <none>        24m
NAME                                STATUS    AGE
default                             Active    24d
kube-node-lease                     Active    24d
kube-public                         Active    24d
kube-system                         Active    24d
local-path-storage                  Active    24d
test-portfolio                      Active    46m

```

Ces commandes permettent de vérifier que le backend, l'autoscaler, la politique réseau et le namespace sont bien actifs et fonctionnels.

Résumé

Ce projet montre comment déployer une application backend scalable, sécurisée et résiliente dans Kubernetes, avec gestion fine des ressources et de la sécurité réseau.